



University of the
West of England

ADC Data Processor Development Logbook

UFMFGT-15-1 | Programming For Engineers

by

Mohammed ALMERI

Academic Year 2025 - 2026

Github Repository:

[M-ALMERI/adc_data_proc](https://github.com/M-ALMERI/adc_data_proc)

Table of Content

Table of Content.....	2
1. Project Overview	3
2. Project Development.....	4
2.1. The Binary Layout.....	4
2.2. Reading the Dataset.....	4
2.3. Data Processing	4
2.4. Analysis Report	4
3. Understanding the Dataset.....	5
4. Program Design Decisions.....	6
5. Memory Usage and Data Handling.....	6
6. Program Verification.....	7
7. Problems Encountered During Development	8
8. Evaluation & Results	9
9. Conclusion:	9
References:.....	10

1. Project Overview

The aim of this project was to create a command line application that gave the capability to analyze binary records created with the help of a four-channel ADC. This software accepts the dataset as an argument, checks the format of the specified records, reads all the messages in the file, processes the data, identifies the abnormal situation, and writes two reports as a result of the analysis.

In contrast with text formats like CSV, the bytes given in the project cannot be opened for a user in the traditional way. There is a specific meaning of each byte and the only correct way to interpret its content is to open it according to the file specification defined in the coursework. Therefore, one of the priorities during the development was making sure that the binary structure used in the application corresponds to the one applied in the file.

2. Project Development

Rather than trying to satisfy each requirement all at once, the software was developed feature by feature, which limited the introduction of errors in the course of the process and facilitated the validation of the functioning of each part before its incorporation into the entire application.

2.1. The Binary Layout

The first step was to review the binary file definition that was provided for this task. One had to pay special attention to the header, which specifies the number of sensor records, the file version, the number of ADC channels, and the sampling rate. These fields are validated before any records are processed.

Then, the project structure was designed. It comprised separate header and source files for each function of the software in order to isolate the reading of the file from the data processing and report generation.

2.2. Reading the Dataset

After the structure for the project was established, functions were developed to read the binary file, check the header and load sensor information into memory.

The first problem that I faced at this stage was that of structure alignment. The binary files in the dataset use exactly sixteen bytes, whereas the structure in the program contained padding bytes. Because of that, the dataset was not correctly interpreted by the program and an unexpected end of the file error appeared. The problem was solved by packing the structure.

2.3. Data Processing

After successfully reading the dataset, it was necessary to process obtained measurements. The values obtained from ADC were converted into voltages using the conversion formula which I got in my coursework. After that, each channel was analysed separately to get statistical data, such as average voltage, RMS voltage, minimum value, maximum value and standard deviation.

2.4. Analysis Report

The last milestone has to do with the creation of user-friendly reports that sum up the analysis performed.

The first report was created right after analyzing the dataset. It shows the complete information about the dataset and gives statistical data for each channel. The second report presents the

information about all the problems detected with their timestamps, channel codes, and the reason for being detected.

3. Understanding the Dataset

It was important to know everything that each record contains and how that information is related to the practical real ADC before performing an analysis. The ADC used for this project is a twelve-bit ADC. A twelve-bit ADC has the capacity of generating potential values from 0 to 4095, which indicate equal-sized levels of potential from 0 volts to the reference voltage of 3.3 volts. Because the values in question are numeric instead of being voltages, it is important to convert them before any processing can be done. The expression used in the complete project in order to convert raw value to voltage is the following one:

$$voltage = (raw_value / 4095.0) * 3.3$$

Using 4095.0 rather than 4095 ensures that floating point division will occur instead of integer division, meaning that small raw numbers will not yield a truncated value of zero. The ADC samples all four channels using time-division multiplexing, so every record contains a channel identifier indicating which channel pertains to which measurement, given that all four channels use the same physical converter.

Furthermore, each record is comprised of a status flag field. Instead of storing numerous Boolean variables separately, various different pieces of information are saved within one byte. The value of each individual bit indicates if a fault or out of tolerance has taken place. Thus, it is needed to use bit operations to evaluate which conditions are active.

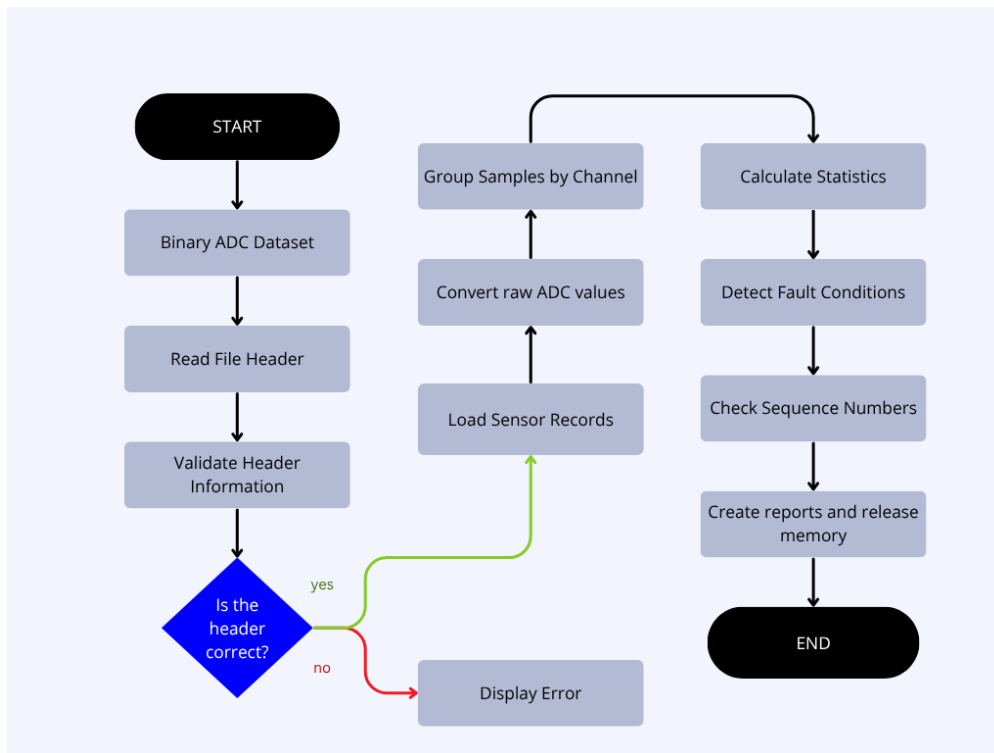


Figure 1: ADC Data Processing Pipeline

The figure above is the data processing pipeline illustrating how binary ADC records are transformed into statistical results and fault reports.

4. Program Design Decisions

Development included several implementation decisions that enhanced the readability of the program while also ensuring its simplicity.

The development was divided into 4 modules. The I/O Module dealt mainly with file input/output and report generation activities. The module on statistics calculated the necessary statistics as the name of the module indicates, and the resources module catered for operations specific to the functionalities of the ADC. The main module of the program only coordinated the processing of the modules but without making calculations.

Another decision made was to separate the structure used in reading binary records from that used in the program. The binary structure closely mirrors the structure of the file being read whilst the internal structure provides for the storage of values such as the voltage which are derived from the binary structure.

A crucial detail that involved extensive consideration was the padding introduced by the compiler. The testing of the binary record structure presented a challenge because of the additional insertion of padding bytes by the compiler. The use of struct packing was beneficial in ensuring that every field was arranged accurately according to its position.

Finally, the analysis involved a fixed-size array which could accommodate a maximum of thirty-two sequence gaps. The analysis will still go through the entire data analysis and recognize all the gaps; however, it has limited the number of records that it will print out.

5. Memory Usage and Data Handling

The utilization of dynamic memory allocation is necessary because there is no way to know how many records there are until the header file has been validated. Once validation is completed, the program will allocate memory using malloc() so that memory must be allocated using the total amount of records multiplied by the amount of memory allocated for a single ADCSample structure.

Once the memory has been allocated, the program checks the pointer against NULL. If allocation of memory was unsuccessful, the program will print an error message and stop execution immediately. Performing this check prevents undefined behavior from appearing in the case the program attempts to access the memory that has not been allocated yet.

Moreover, instead of copying the data arrays between functions, the program provides pointer to the allocated memory. This keeps the program efficient by passing addresses instead of

copied data. A few analysis techniques also utilize pointer arithmetic for moving through the sample data that demonstrates another way of accessing the elements of the arrays.

Following the termination of the program, all the dynamic allocations are cleared via calling the method `free()`. Even though memory is cleared by OS when ending the process, it is wiser to free dynamically allocated memory to eliminate any chances of memory leaks in case of making the program a permanent one.

During the analysis development program, the decision was made only to allow the sequence discontinuity showing only up to thirty-two gaps. Despite the program going through the entire data for any gaps made, only the first thirty-two snags will appear on the report. It is because the analysis data does not have up to two discontinuities making thirty-two gaps in total plenty enough to keep the memory management simply and efficiently done.

6. Program Verification

Testing was done at each stage in the development process instead of testing once everything was completed. This allowed the team to know exactly where things went wrong if the system behaved unexpectedly and minimized the debugging effort involved at a later stage of the project.

The final version of the software was verified using both the original input data and additional input data catering for various error conditions.

Test Case	Expected Bahviour	Result
Valid Dataset	Reports generated successfully	Passed
Missing File	Display file opening error	Passed
Missing command line argument	Display usage information	Passed
Empty file	Reject invalid header	Passed
Truncated file	Detect incomplete dataset	Passed
Invalid magic number	Reject incorrect file format	Passed

The execution proved the success of creating both `results.txt` and `fault_report.txt` without any problem in accordance with the dataset sent. The rest of experiments showed that any invalid inputs are catching correctly and do not lead to system crashes or behavior which can be described as undefined.

7. Problems Encountered During Development

While the system was being built, some small issues arose while different parts of the system were being integrated. Each of these problems had to be approached differently to make a system that could provide the correct functionality.

- The Reading of Binary Recordings

The first issue was the need to make sure the system could read the binary data correctly. Even though the binary file was known, the binary data could not be read correctly due to the fact that its representation in memory was not matching that in the binary file. The way out was to use a special packed structure that was exactly matching the specification of the binary data.

- The Difference Between Binary and Processed Data

At first, the same structure was planned to be used for reading the binary data and storing the processed data. However, during the development process, it became clear that it was much simpler to have two different structures: one for reading the binary records and the other for storing the processed data.

- Formatting the Report

It required quite a few revisions to come up with readable results. The previous versions of the report showed information in a linear manner making it hard to find statistics or fault information in short time. The way the current report displays information makes it more readable, and therefore gives an easy chance to pinpoint any necessary information in the report.

- Setting a limit on the number of sequence gaps reported

There became a need to either allocate memory dynamically or limit the number of sequence gaps stored. But it was decided to fix the buffer size to allow storage of up to thirty-two gaps. The given approach favors simplicity of implementation and allows the reports to contain relevant information.

8. Evaluation & Results

Evaluation & Findings

Once the implementation was done, the program went through an evaluation phase where it was tested with the original ADC dataset and different modified test files. The successful testing proved that the program is capable of opening the binary file, verifying the header, loading the records, converting raw ADC readings into voltages, calculating statistics of channels and creating output reports.

The report of findings included the main findings of every channel such as average voltage, root mean square voltage, minimum and maximum value, standard deviation, temperature data and fault counts. The fault report presented the details of the faulty records and included the timestamp of the record, the number of the channel, the voltage value, flags and the reason for being faulty.

The program was also tested with invalid input files such as a missing file, an empty binary file, a truncated file and a file with an invalid magic number. The progress of the program while dealing with these files was smooth with proper error message in each case thus ensuring that the program did not crash at any point.

Overall, the program has successfully met all objectives of the coursework.

9. Conclusion

In this project, a modular C application was developed for the purpose of analyzing binary data from ADC sensors. The application developed resulted in the validation of the input file, conversion of raw ADC data into voltage values, calculation of the statistics for each input channel, detection of any faults within the input signal, and also the generation of necessary output reports.

During the process of development, it was necessary to conduct tests in order to check normal operation of the application and how it handles errors. The project was aimed at fulfilling all coursework requirements related to using dynamic memory allocation, binary file processing, modular programming, pointer usage, and structured testing techniques.

10. References

1. University of the West of England - UFMFGT-15-1 Programming for Engineers: Resit Coursework Brief - Embedded Sensor Data Processor.
2. Microchip Technology Inc. (2024) ADC (Analog-to-Digital Converter) Fundamentals. Available at: <https://www.microchip.com/> (Accessed: 5 July 2026).
3. Microsoft Learn - malloc (C). Available at: <https://learn.microsoft.com/cpp/c-runtime-library/reference/malloc> (Accessed: 6 July 2026).
4. IBM (2024) C Language Reference. Available at: <https://www.ibm.com/docs/> (Accessed: 6 July 2026).
5. Git (2026) Git Documentation. Available at: <https://git-scm.com/doc> (Accessed: 8 July 2026).